TECNICATURA SUPERIOR EN CIENCIA DE DATOS E INTELIGENCIA ARTIFICIAL

Centro Educativo Paulo Freire — U.E.G.P. N° 165

CLASE 14

Aprendizaje No Supervisado (Clustering I)

K-Means: descubriendo estructuras ocultas en los datos

Datos de la clase

Bloque teórico: **40 minutos** · Bloque práctico: **40 minutos** · Total: **80 minutos**

Bibliografía de referencia: *An Introduction to Statistical Learning* (James et al.) y *Python Data Science Handbook* (VanderPlas).

Objetivo de la clase

Comprender cómo encontrar subgrupos homogéneos y naturales dentro de un conjunto de datos cuando no existe una variable objetivo Y que supervise el aprendizaje, e introducir K-Means como el algoritmo de clustering más representativo para lograrlo.

1. ¿Qué es el Clustering?

Clustering (agrupamiento) es una técnica de **aprendizaje no supervisado** que consiste en particionar un conjunto de observaciones en grupos —llamados clusters— de modo que:

- **Similitud interna:** las observaciones dentro de un mismo cluster son muy parecidas entre sí.
- **Disimilitud externa:** las observaciones de clusters distintos son lo más diferentes posible entre sí.

La "similitud" se mide numéricamente mediante una métrica de distancia (habitualmente la distancia euclídea), aplicada sobre las variables predictoras X de cada observación.

1.1 Diferencia con Clasificación

Aspecto	Clasificación (supervisado)	Clustering (no supervisado)
Variable objetivo Y	Conocida de antemano (etiquetas)	No existe
Ejemplo	"Spam" vs. "No Spam"	Segmentos de clientes desconocidos
Tarea del algoritmo	Aprender la función X → Y	Inferir la estructura de grupos a partir de X
Evaluación	Accuracy, precision, recall, etc. (hay verdad de referencia)	Métricas internas (inercia, silhouette); no hay verdad de referencia

Esta ausencia de etiquetas es la diferencia conceptual central: en clustering el algoritmo debe descubrir por sí mismo qué observaciones "van juntas", sin que nadie le indique previamente la respuesta correcta.

2. Fundamento Matemático de K-Means

2.1 Partición no solapada

K-Means asume que cada observación pertenece exactamente a un único cluster. Formalmente, dados K clusters C_1, C_2, …, C_K, se deben cumplir dos condiciones:

$$C_1 \cup C_2 \cup \ldots \cup C_K = \{1, \ldots, n\}$$

Es decir, la unión de todos los clusters cubre el conjunto completo de observaciones (todas quedan asignadas a algún grupo), y:

$$C_k \cap C_{k'} = \emptyset \quad \text{para todo } k \neq k'$$

Es decir, ningún par de clusters distintos comparte observaciones. No existe superposición: no hay pertenencia parcial ni simultánea a dos grupos.

2.2 Función objetivo: minimizar la variación intra-cluster

El objetivo de K-Means es encontrar la partición que **minimiza la variación total dentro de los clusters** (llamada within-cluster variation o **inercia**):

$$\min_{C_1, \ldots, C_K} \sum_{k=1}^{K} W(C_k)$$

Donde W(C_k) mide cuánto varían entre sí las observaciones dentro del cluster k. Cuanto menor sea esta variación, más compactos y homogéneos serán los grupos.

Definición de W(C_k)

La forma más habitual de definir W(C_k) es a partir de la distancia euclídea al cuadrado entre todos los pares de observaciones del cluster:

$$W(C_k) = \frac{1}{|C_k|} \sum_{i,\, i' \in C_k} \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2$$

> **Nota**
> |C_k| es la cantidad de observaciones en el cluster k. El doble sumatorio recorre todos los pares posibles (i, i') dentro del mismo cluster, y j recorre las p variables (columnas) del dataset.

Forma equivalente (la que efectivamente usa el algoritmo)

Puede demostrarse algebraicamente que esa doble suma sobre todos los pares de puntos equivale a la suma de distancias al cuadrado entre cada punto y el centroide (la media) de su cluster:

$$W(C_k) = 2 \sum_{i \in C_k} \sum_{j=1}^{p} (x_{ij} - \bar{x}_{kj})^2$$

Donde el centroide del cluster k, x̄_k, es simplemente el promedio de todas las observaciones asignadas a ese cluster, en cada variable j:

$$\bar{x}_{kj} = \frac{1}{|C_k|} \sum_{i \in C_k} x_{ij}$$

Por qué importa esta equivalencia: calcular todos los pares de distancias es computacionalmente costoso (crece de forma cuadrática con el tamaño del cluster). Calcular la distancia de cada punto a un único centroide es mucho más eficiente — y es exactamente lo que hace el algoritmo en la práctica.

2.3 Distancia euclídea

Tanto la asignación de puntos a clusters como el cálculo de la inercia se basan en la distancia euclídea entre una observación y un centroide μ_k:

$$d(x_i, \mu_k) = \sqrt{\sum_{j=1}^{p} (x_{ij} - \mu_{kj})^2}$$

Y la inercia total del modelo —la métrica que se reporta y se compara entre distintas corridas o distintos valores de K— se expresa como:

$$\text{Inercia} = \sum_{k=1}^{K} \sum_{i \in C_k} \|x_i - \mu_k\|^2$$

3. El Algoritmo Iterativo, Paso a Paso

K-Means es un algoritmo iterativo de tipo Esperanza-Maximización (EM): alterna entre asignar puntos a clusters y recalcular los centroides, hasta estabilizarse.

Paso 0 — Inicialización

Se elige el número de clusters K (un hiperparámetro que el analista debe definir de antemano) y se ubican K centroides iniciales en el espacio de las variables — normalmente eligiendo K observaciones al azar del propio dataset.

Paso 1 — Asignación ("Paso E")

Cada observación se asigna al centroide más cercano, según la distancia euclídea definida en la sección anterior. Al finalizar este paso, cada punto pertenece a exactamente un cluster.

Paso 2 — Actualización ("Paso M")

Se recalcula la posición de cada centroide como el promedio (media aritmética) de todos los puntos que quedaron asignados a ese cluster en el paso anterior. Los centroides "se mueven" hacia el centro de masa de su grupo.

Paso 3 — Convergencia

Se repiten los pasos 1 y 2 hasta que ocurra una de estas dos condiciones:

- Los centroides dejan de cambiar de posición entre una iteración y la siguiente (o el cambio es menor a una tolerancia mínima).
- Se alcanza un número máximo de iteraciones definido de antemano.

Propiedad clave: en cada iteración la inercia total nunca aumenta. El algoritmo garantiza una mejora monótona (o estabilidad) del objetivo, lo cual asegura que siempre converge — aunque, como se ve en la sección 4, no necesariamente al mejor resultado posible.

> **Resumen visual del ciclo**
> Inicializar K centroides → Asignar cada punto al centroide más cercano → Recalcular centroides como la media de su grupo → ¿Cambiaron los centroides? Si sí, repetir; si no, terminar.

4. Consideraciones Críticas del Algoritmo

4.1 Óptimos locales y múltiples inicios

K-Means **no garantiza encontrar el mínimo global** de la función objetivo. El resultado final depende de dónde se ubicaron los centroides iniciales: distintas inicializaciones pueden converger a particiones distintas, algunas mejores que otras (óptimos locales).

Solución práctica: ejecutar el algoritmo múltiples veces con inicializaciones aleatorias distintas, y quedarse con la corrida que logró la menor inercia. En scikit-learn esto se controla con el parámetro

```
n_init=20   # o n_init=50 — número de inicializaciones distintas a probar
```

Cuantas más inicializaciones, mayor la probabilidad de acercarse al óptimo global — a costa de mayor tiempo de cómputo.

4.2 Sensibilidad a la escala de las variables

Como K-Means se basa en distancia euclídea, las variables con **magnitudes grandes dominan el cálculo de distancia** frente a variables con magnitudes pequeñas, aunque estas últimas sean igual de relevantes conceptualmente.

> **Ejemplo**
> Si una variable es "ingreso anual" (valores del orden de 10^6) y otra es "edad" (valores entre 0 y 100), la distancia euclídea estará dominada casi por completo por el ingreso — la edad prácticamente no influirá en la asignación de clusters, sin importar cuánto varíe.

Por eso es indispensable estandarizar previamente todas las variables, llevándolas a media 0 y desvío estándar 1, antes de aplicar K-Means:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_escalado = scaler.fit_transform(X)   # X: DataFrame con las variables predictoras
```

4.3 Determinación de K — el Método del Codo (Elbow Method)

A diferencia de otros hiperparámetros, K no se puede "aprender" de los datos: debe elegirse antes de ejecutar el algoritmo. El Método del Codo es la heurística más usada para elegirlo:

1. Se entrena el modelo K-Means para un rango de valores de K (por ejemplo, de 1 a 10).
2. Para cada valor de K, se registra la inercia final del modelo (el atributo inertia_ en scikit-learn).
3. Se grafica la inercia (eje Y) en función de K (eje X).
4. Se busca el "codo" de la curva: el punto donde agregar más clusters deja de reducir significativamente la inercia.

La inercia siempre disminuye a medida que K aumenta (en el caso extremo, K = n produce inercia 0, un cluster por punto). El objetivo del método no es minimizar la inercia a cualquier costo, sino encontrar el punto de rendimientos decrecientes: el K más pequeño que ya captura la mayor parte de la estructura de los datos.

```
inercias = []
rango_k = range(1, 11)

for k in rango_k:
    modelo = KMeans(n_clusters=k, n_init=20, random_state=42)
    modelo.fit(X_escalado)
    inercias.append(modelo.inertia_)
```

```
plt.plot(rango_k, inercias, marker='o')
plt.xlabel('Número de clusters (K)')
plt.ylabel('Inercia')
plt.title('Método del Codo')
```

Para tener en cuenta antes del bloque práctico

K-Means requiere definir K de antemano y es sensible a outliers y a la forma de los clusters (asume grupos aproximadamente esféricos y de tamaño similar). Estas limitaciones —junto con alternativas como clustering jerárquico— se retoman en la Clase 15.